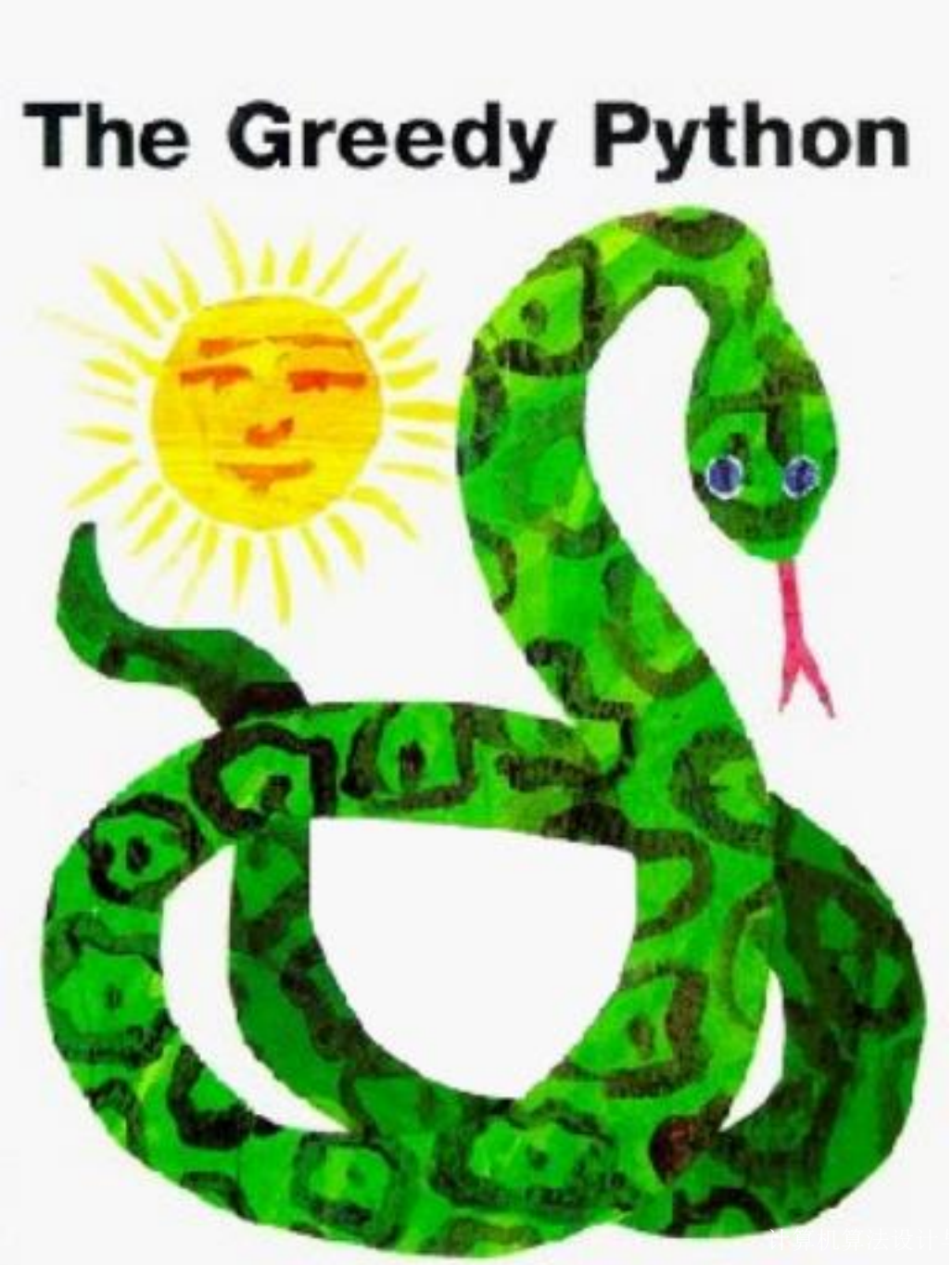


算法分析与设计
Analysis and Design of Computer Algorithms
贪心算法

李纲

宁波大学
信息科学与工程学院

- **学习要点：**
- 理解贪心算法的概念。
- 掌握贪心算法的基本要素
- （1）最优子结构性质
- （2）贪心选择性质
- 理解贪心算法与动态规划算法的差异
- 理解贪心算法的一般理论
- 通过应用范例学习贪心设计策略。
- （1）活动安排问题；
- （2）最优装载问题；
- （3）哈夫曼编码；
- （4）单源最短路径；
- （5）最小生成树；
- （6）多机调度问题。

顾名思义，贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是整体最优的。

虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路经问题，最小生成树问题等。在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

局部最优->整体最优

4.1 活动安排问题

活动安排问题就是要在所给的活动集合中：

选出最大的相容活动子集合，

是可以用贪心算法有效求解的很好例子。该问题要求高效地安排一系列争用某一公共资源的活动。贪心算法提供了一个简单、漂亮的方法使得尽可能多的活动能兼容地使用公共资源。

不一定是最大利用率

4.1 活动安排问题

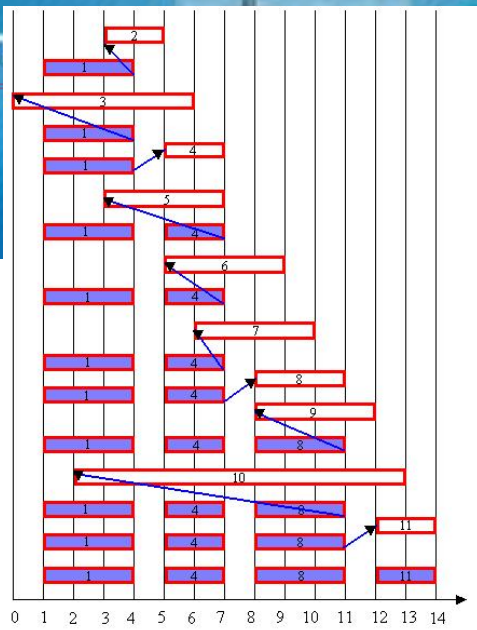
设有n个活动的集合E={1,2,…,n}，其中每个活动都要求使用同一资源，如演讲会场等，而在同一时间内只有一个活动能使用这一资源。【选出最大的相容活动子集合】

每个活动i都有一个要求使用该资源的起始时间si和一个结束时间fi,且si <fi 。

如果选择了活动i，则它在半开时间区间[si, fi)内占用资源。

若区间[si, fi)与区间[sj, fj)不相交，则称活动i与活动j是相容的。也就是说，当si≥fj或sj≥fi时，活动i与活动j相容。

4.1 活动安排问题



例：设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

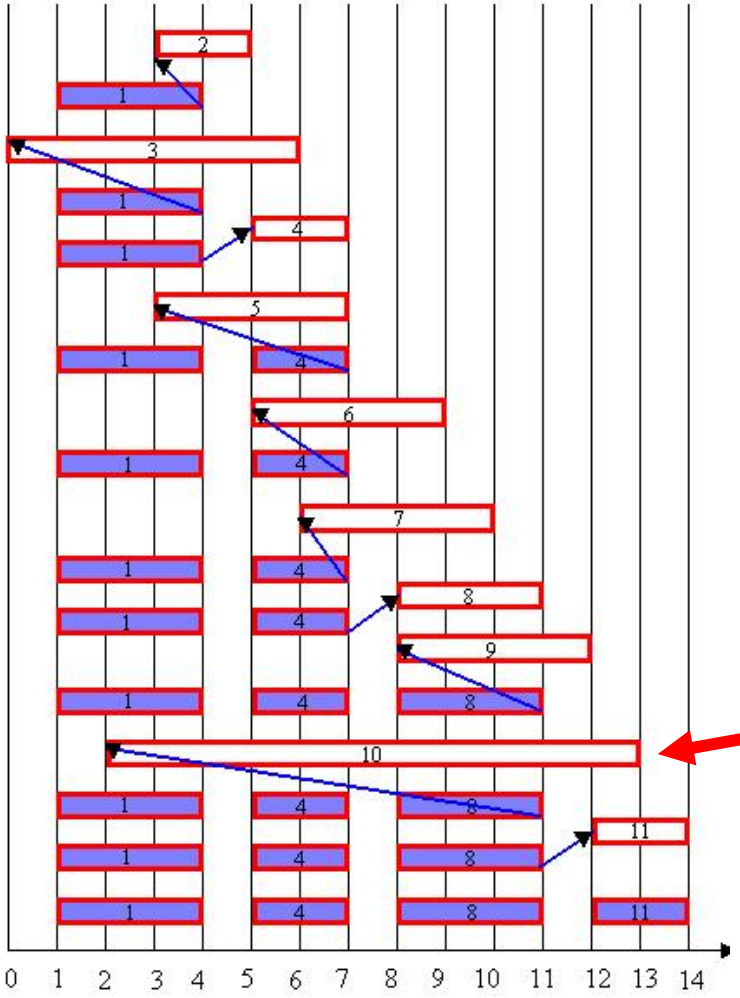
i	1	2	3	4	5	6	7	8	9	10	11
S[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

4.1 活动安排问题

由于输入的活动以其完成时间的**非减序**排列，所以算法**greedySelector**每次总是选择**具有最早完成时间**的相容活动加入集合A中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化，以便安排尽可能多的相容活动。**

算法**greedySelector**的效率极高。当输入的活动已按结束时间的非减序排列，算法只需**O(n)**的时间安排n个活动，使最多的活动能相容地使用公共资源。如果所给出的活动未按非减序排列，可以用**O(nlogn)**的时间重排。

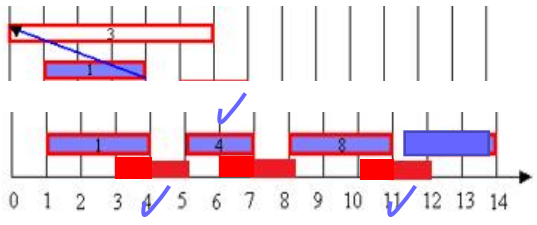
4.1 活动安排问题



算法greedySelector 的计算过程如左图所示。图中每行相应于算法的一次迭代。阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正在检查相容性的活动。

动态规划怎么做?

最大利用率怎么做?



按开始和按间隔特例：
<:按开始时间
<:按最短时间（红色）
红色新增，选择✓

4.1 活动安排问题

下面给出解活动安排问题的贪心算法GreedySelector：

```
template<class Type>
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
      A[1]=true;
      int j=1;
      for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
       }
}
```

各活动的起始时间和结束时间存储于数组s和f中且按结束时间的非减序排列

4.1 活动安排问题

若被检查的活动i的开始时间Si小于最近选择的活动j的结束时间fi，则不选择活动i，否则选择活动i加入集合A中。

贪心算法并不总能求得问题的**整体最优解**。但对于活动安排问题，贪心算法greedySelector却总能求得的整体最优解，即它最终所确定的相容活动集合A的规模最大。这个结论可以用数学归纳法证明。

适不适合用贪心算法

4.2 贪心算法的基本要素

本节着重讨论可以用贪心算法求解的问题的一般特征。

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢?这个问题很难给予肯定的回答。

但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：

贪心选择性质和**最优子结构性质**。

4.2 贪心算法的基本要素

1、贪心选择性质

所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

动态规划算法通常以**自底向上**的方式解各子问题，

而**贪心算法**则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。致问题的整体最优解。

4.2 贪心算法的基本要素

2、最优子结构性质

当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

4.2 贪心算法的基本要素

3、贪心算法与动态规划算法的差异

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解?是否能用动态规划算法求解的问题也能用贪心算法求解?

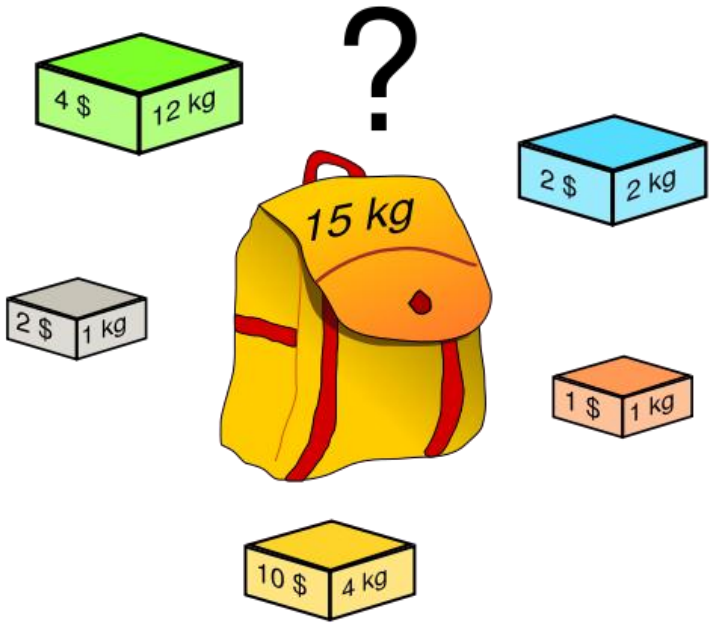
下面研究2个经典的**组合优化问题**，并以此说明贪心算法与动态规划算法的主要差别。

4.2 贪心算法的基本要素

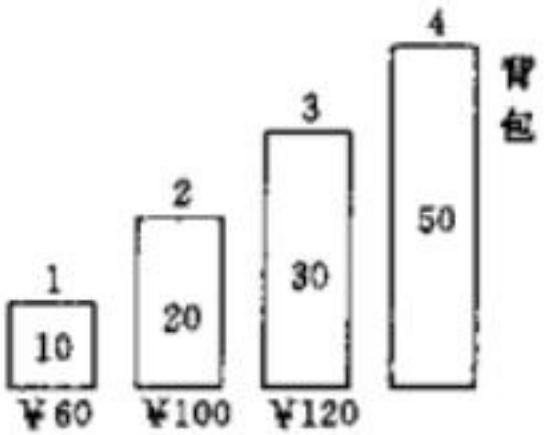
0-1背包问题：

给定n种物品和一个背包。物品i的重量是Wi，其价值为Vi，背包的容量为C。应如何选择装入背包的物品，使得装入背包中物品的总价值最大?

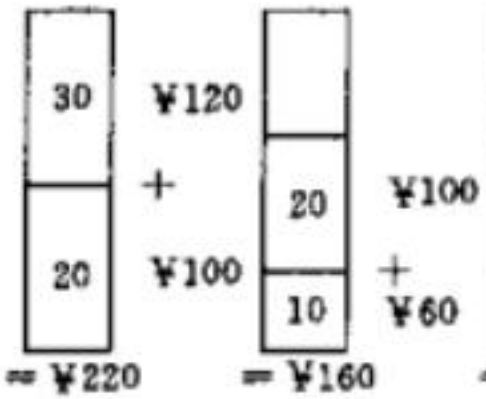
在选择装入背包的物品时，对每种物品i只有2种选择，即装入背包或不装入背包。不能将物品i装入背包多次，也不能只装入部分的物品i。



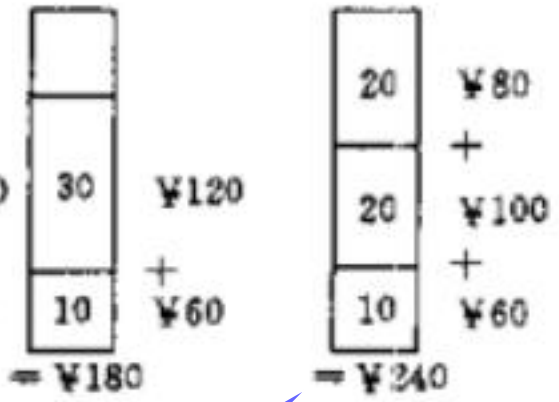
4.2 贪心算法的基本要素



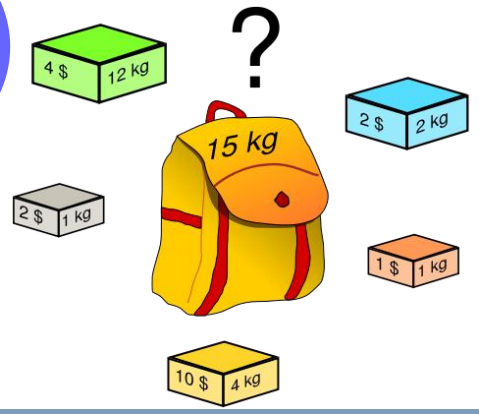
每件只有一个



0、1背包



普通背包3号选用了2/3



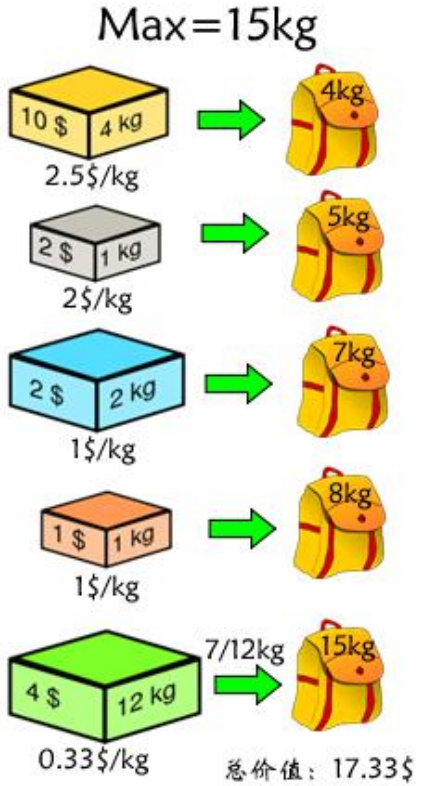
4.2 贪心算法的基本要素

- **背包问题：**

与0-1背包问题类似，所不同的是在选择物品i装入背包时，**可以选择物品i的一部分**，而不一定要全部装入背包，1≤i≤n。

这2类问题都具有**最优子结构**性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。

P107例子



4.2 贪心算法的基本要素

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值Vi/Wi，然后，依贪心选择策略，将尽可能多的**单位重量价值最高**的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过C，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

具体算法可描述如下页：

4.2 贪心算法的基本要素

```
void Knapsack(int n,float M,float v[],float w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++) x[i]=0;
    float c=M;
    for (i=1;i<=n;i++) {
      if (w[i]>c) break;
      x[i]=1;
      c-=w[i];
      }
    if (i<=n) x[i]=c/w[i];   //空间不够只取部分
}
```

算法knapsack的主要计算时间在于将各种物品**依其单位重量的价值从大到小排序**。因此，算法的计算时间上界为O（nlogn）。

为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。

4.2 贪心算法的基本要素

对于**0-1背包问题**，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的背包空间使每公斤背包空间的价值降低了。事实上，在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。由此就导出许多互相重叠的子问题。这正是该问题可用**动态规划算法**求解的另一重要特征。

实际上也是如此，动态规划算法的确可以有效地解0-1背包问题。

贪心：背包的解肯定好于01背包，有何用处?

4.3 最优装载

有一批集装箱要装上一艘载重量为c的轮船。其中集装箱i的重量为Wi。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

1、算法描述

最优装载问题可用贪心算法求解。采用重量最轻者先装的贪心选择策略，可产生最优装载问题的最优解。具体算法描述如下页。

我们发现，贪心算法可能会得到一个上限

4.3 最优装载

```
template<class Type>
void Loading(int x[],  Type w[], Type c, int n)
{
        int *t = new int [n+1];
        Sort(w, t, n);
        for (int i = 1; i <= n; i++) x[i] = 0;
        for (int i = 1; i <= n && w[t[i]] <= c; i++) {x[t[i]] = 1; c -= w[t[i]];}
}
```

4.3 最优装载

2、贪心选择性质

可以证明最优装载问题具有贪心选择性质。

3、最优子结构性质

最优装载问题具有最优子结构性质。

由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法**loading**的正确性。

算法**loading**的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 **O(nlogn)**。

4.4 哈夫曼编码

哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%～90%之间。哈夫曼编码算法用字符在文件中出现的频率表来建立一个用0，1串表示各字符的最优表示方式。

(1)给出现频率高的字符较短的编码，(2)出现频率较低的字符以较长的编码，可以大大缩短总码长。

1、前缀码

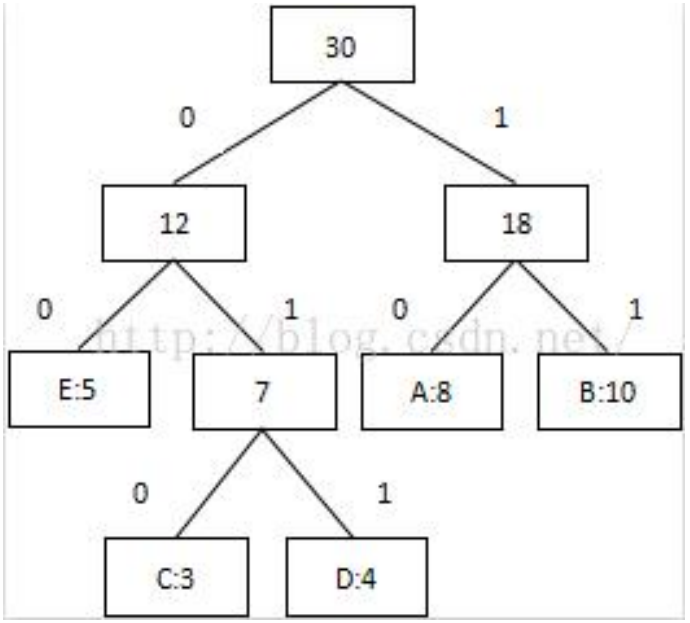
对每一个字符规定一个0,1串作为其代码，并要求任一字符的代码都不是其它字符代码的前缀。这种编码称为**前缀码**。

4.4 哈夫曼编码

哈夫曼编码

BABACAC ADADABB CBABEBE DDAB

字符	次数	编码
B	10	11
A	8	10
C	3	010
D	4	011
E	5	00



http://blog.csdn.net/

4.4 哈夫曼编码

编码的前缀性质可以使译码方法非常简单。

*表示**最优前缀码**的二叉树总是一棵**完全二叉树()**，即树中任一结点都有2个儿子结点。*

平均码长定义为：$$B(T)=\sum_{c\in C} f(c)d_T(c)$$

使平均码长达到最小的前缀码编码方案称为给定编码字符集C的**最优前缀码**。

若一棵二叉树至多只有最下面的两层上结点的度数可以小于2，并且最下一层上的结点都集中在该层最左边的若干位置上，则此二叉树称为完全二叉树

4.4 哈夫曼编码

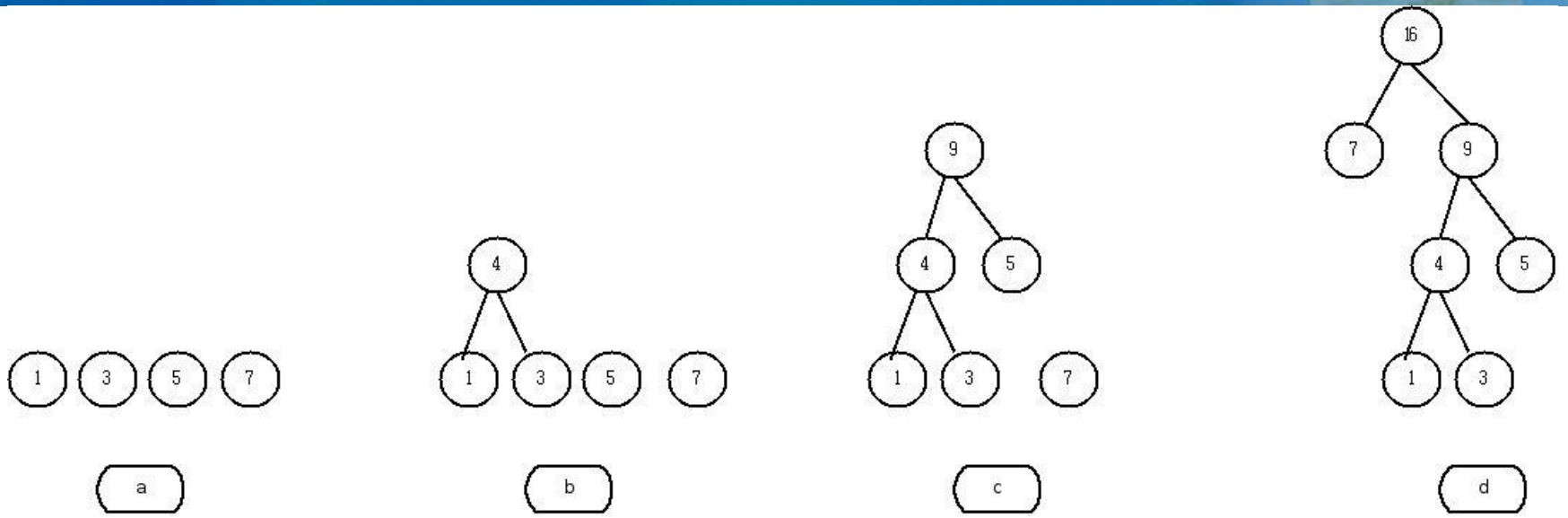
2、构造哈夫曼编码

哈夫曼提出构造最优前缀码的贪心算法，由此产生的编码方案称为**哈夫曼编码**。

哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树T。

算法以|C|个叶结点开始，执行|C| - 1次的“合并”运算后产生最终所要求的树T。

4.4 哈夫曼编码



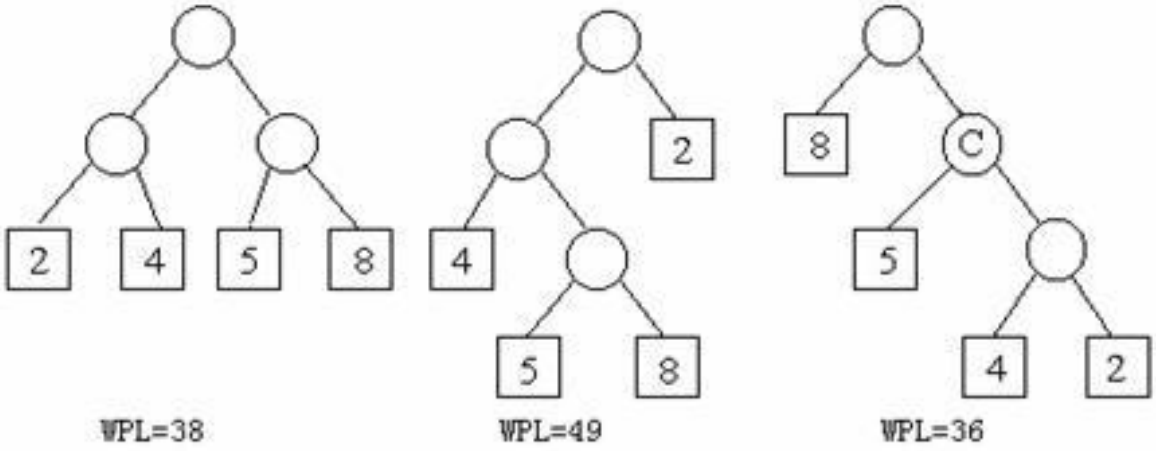


图6.21 具有相同叶子不同带权路径长度的二叉树

4.4 哈夫曼编码

在书上给出的算法huffmanTree中，编码字符集中每一字符c的频率是f(c)。**以f为键值的优先队列Q**用在**贪心选择**时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的2棵树的频率之和，并将新树插入优先队列Q。经过n - 1次的合并后，优先队列中只剩下一棵树，即所要求的树T。

算法huffmanTree用最小堆实现优先队列Q。初始化优先队列需要O(n)计算时间，由于最小堆的removeMin和put运算均需O(logn)时间，n - 1次的合并总共需要O(nlogn)计算时间。因此，关于n个字符的哈夫曼算法的**计算时间**为O(nlogn) 。

4.4 哈夫曼编码

3、哈夫曼算法的正确性

要证明哈夫曼算法的正确性，只要证明最优前缀码问题具有**贪心选择性质**和**最优子结构性质。**

(1)贪心选择性质

(2)最优子结构性质

4.5 单源最短路径

给定带权有向图G =(V,E)，其中每条边的权是非负实数。另外，还给定V中的一个顶点，称为**源**。现在要计算从源到所有其它各顶点的**最短路长度**。这里路的长度是指路上各边权之和。这个问题通常称为**单源最短路径问题**。

1、算法基本思想

Dijkstra算法是解单源最短路径问题的贪心算法。

4.5 单源最短路径

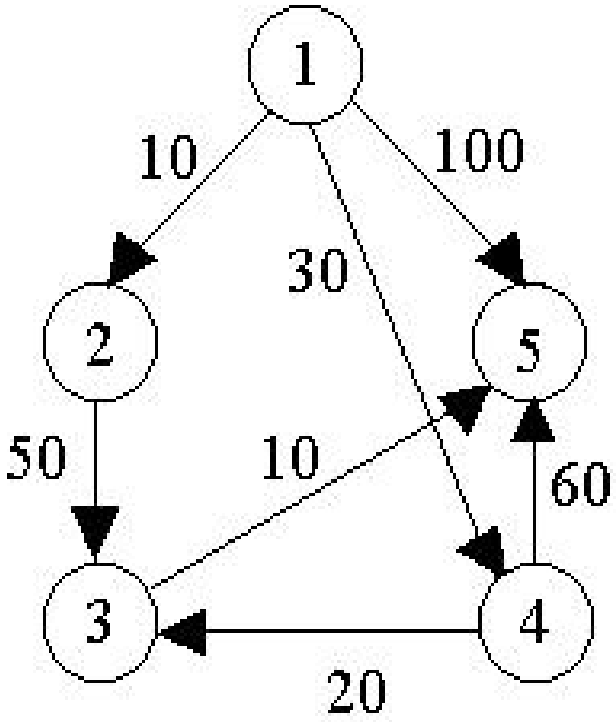
其**基本思想**是，设置顶点集合S并不断地作**贪心选择**来扩充这个集合。一个顶点属于集合S当且仅当从源到该顶点的最短路径长度已知。

初始时，S中仅含有源。设u是G的某一个顶点，把从源到u且中间只经过S中顶点的路称为从源到u的特殊路径，并用数组dist记录当前每个顶点所对应的最短特殊路径长度。

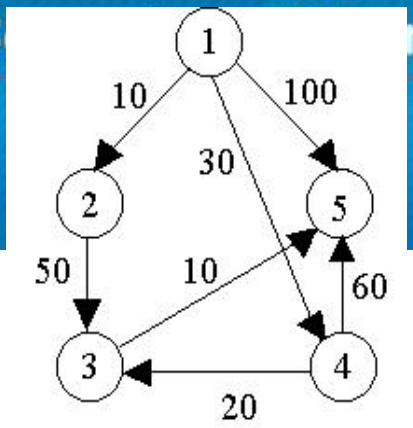
Dijkstra算法每次从V-S中取出具有最短特殊路长度的顶点u，将u添加到S中，同时对数组dist作必要的修改。一旦S包含了所有V中顶点，dist就记录了从源到所有其它顶点之间的最短路径长度。

4.5 单源最短路径

例如，对右图中的有向图，应用Dijkstra算法计算从源顶点1到其它顶点间最短路径的过程列在下页的表中。



4.5 单源最短路径



Dijkstra算法的迭代过程：

取最短的加入

迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

4.5 单源最短路径

2、算法的正确性和计算复杂性

(1)贪心选择性质

(2)最优子结构性质

(3)计算复杂性

对于具有n个顶点和e条边的带权有向图，如果用带权邻接矩阵表示这个图，那么Dijkstra算法的主循环体需要时间。这个循环需要执行n-1次，所以完成循环需要$O(n^2)$时间。算法的其余部分所需要时间不超过$O(n^2)$。

4.6 最小生成树

设G =(V,E)是无向连通带权图，即一个**网络**。E中每条边(v,w)的权为c[v][w]。

如果G的子图G'是一棵包含G的所有顶点的**树**，则称G'为G的生成树。生成树上各边权的总和称为该生成树的**耗费**。在G的所有生成树中，耗费最小的生成树称为G的**最小生成树**。

网络的最小生成树在实际中有广泛应用。**例如**，在设计通信网络时，用图的顶点表示城市，用边(v,w)的权c[v][w]表示建立城市v和城市w之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。

4.6 最小生成树

1、最小生成树性质

用贪心算法设计策略可以设计出构造最小生成树的有效算法。本节介绍的构造最小生成树的**Prim算法**和**Kruskal算法**都可以看作是应用贪心算法设计策略的例子。尽管这2个算法做贪心选择的方式不同，它们都利用了下面的**最小生成树性质**：

设G=(V,E)是连通带权图，U是V的真子集。如果(u,v)∈E，且u∈U，v∈V-U，且在所有这样的边中，(u,v)的权c[u][v]最小，那么一定存在G的一棵最小生成树，它以(u,v)为其中一条边。这个性质有时也称为**MST性质**。

4.6 最小生成树

2、Prim算法

设G=(V,E)是连通带权图，V={1,2,…,n}。

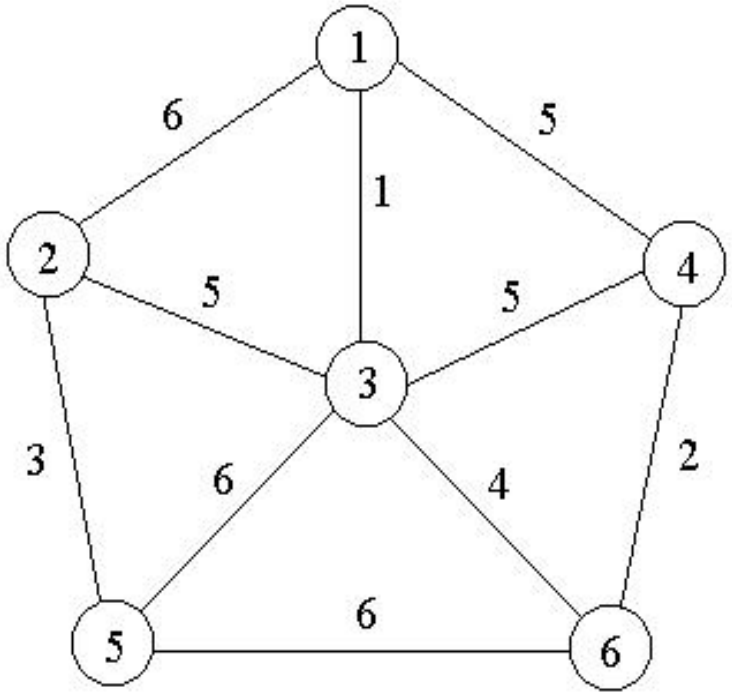
构造G的最小生成树的Prim算法的**基本思想**是：首先置S={1}，然后，只要S是V的真子集，就作如下的**贪心选择**：选取满足条件i∈S，j∈V-S，且c[i][j]最小的边，将顶点j添加到S中。这个过程一直进行到S=V时为止。

在这个过程中选取到的所有边恰好构成G的一棵**最小生成树。**

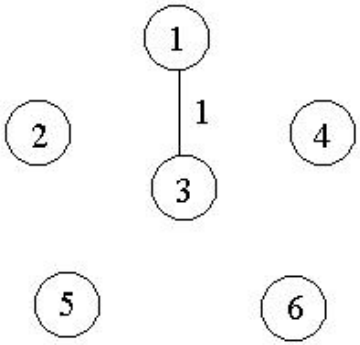
4.6 最小生成树

利用最小生成树性质和数学归纳法容易证明，上述算法中的**边集合T始终包含G的某棵最小生成树中的边**。因此，在算法结束时，T中的所有边构成G的一棵最小生成树。

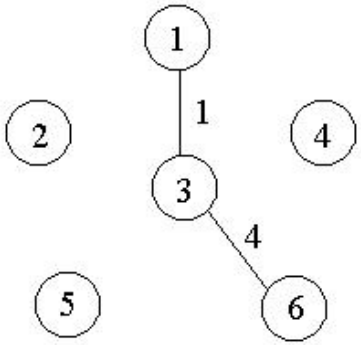
例如，对于右图中的带权图，按**Prim算法**选取边的过程如下页图所示。



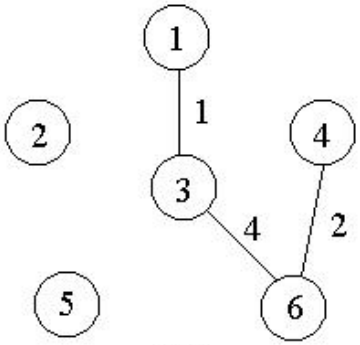
4.6 最小生成树



(a)

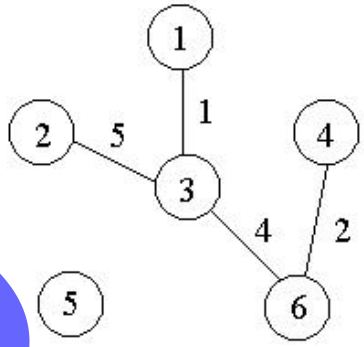


(b)

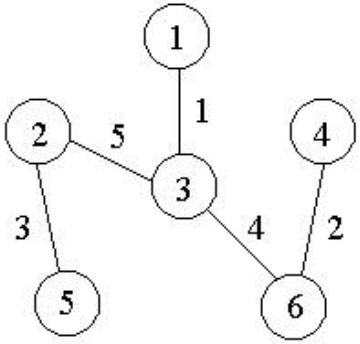


(c)

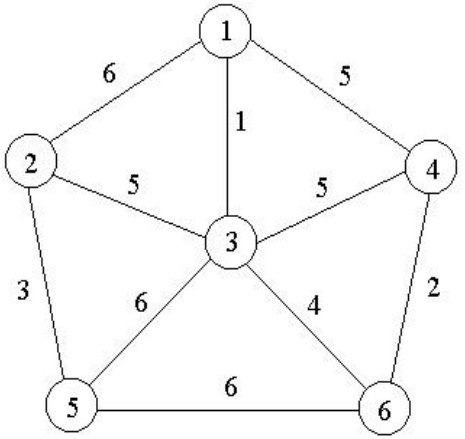
每次选取能加入连接最小的边



(d)



(e)



4.6 最小生成树

在上述Prim算法中，还应当考虑**如何有效地找出满足条件i∈S,j∈V-S，且权c[i][j]最小的边(i,j)**。实现这个目的的较简单的办法是设置2个数组closest和lowcost。

在Prim算法执行过程中，先找出V-S中使lowcost值最小的顶点j，然后根据数组closest选取边(j,closest[j])，最后将j添加到S中，并对closest和lowcost作必要的修改。

用这个办法实现的Prim算法所需的**计算时间**为$O(n^2)$

4.6 最小生成树

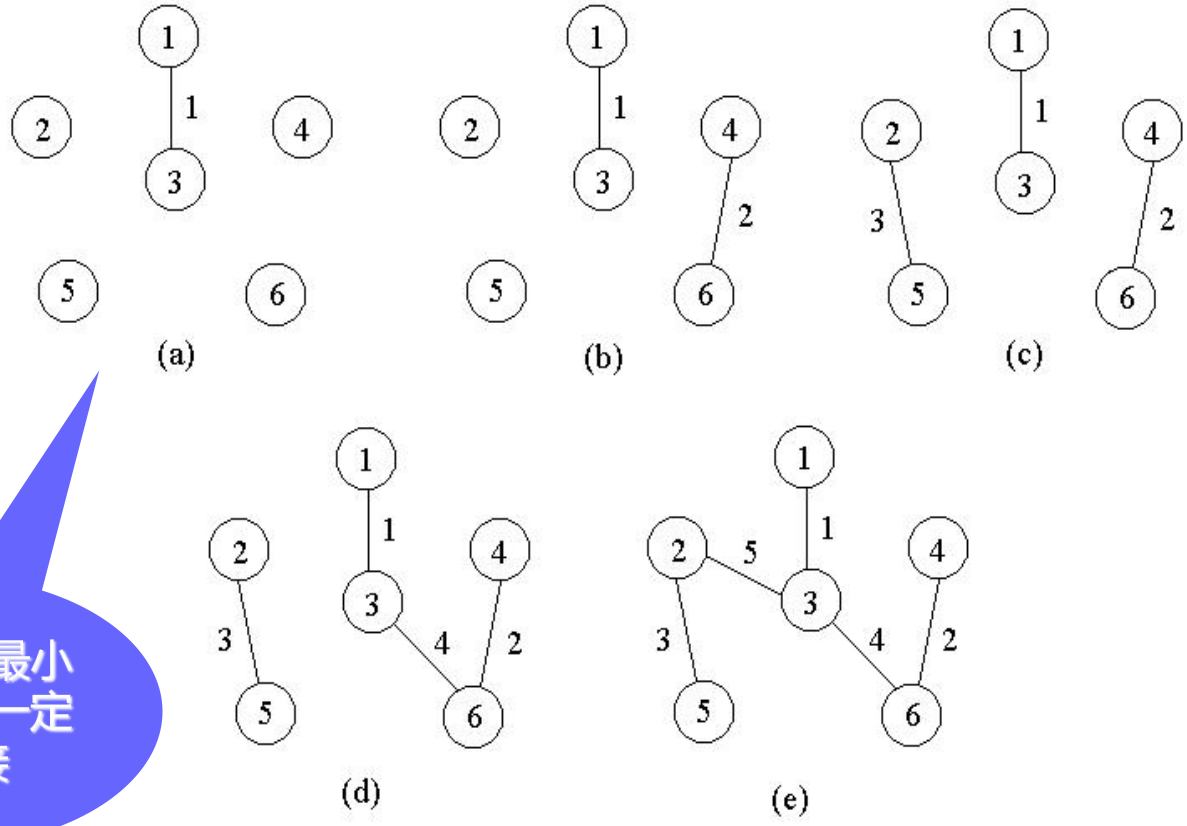
3、Kruskal算法

Kruskal算法构造G的最小生成树的**基本思想**是，首先将G的n个顶点看成n个孤立的连通分支。将所有的边按权从小到大排序。然后从第一条边开始，依边权递增的顺序查看每一条边，并按下述方法连接2个不同的连通分支：当查看到第k条边(v,w)时，如果端点v和w分别是当前2个不同的连通分支T1和T2中的顶点时，就用边(v,w)将T1和T2连接成一个连通分支，然后继续查看第k+1条边；如果端点v和w在当前的同一个连通分支中，就直接再查看第k+1条边。

这个过程一直进行到只剩下一个连通分支时为止。

4.6 最小生成树

例如，对前面的连通带权图，按Kruskal算法顺序得到的最小生成树上的边如下图所示。



4.6 最小生成树

关于**集合的一些基本运算**可用于实现Kruskal算法。

按权的递增顺序查看等价于对**优先队列**执行**removeMin**运算。可以用**堆**实现这个优先队列。

对一个由连通分支组成的集合不断进行修改，需要用到抽象数据类型**并查集UnionFind**所支持的基本运算。

当图的边数为e时，Kruskal算法所需的**计算时间**是 $O(e\log e)$ 。当 $e=o(n^2)$ 时，Kruskal算法比Prim算法差，但当 $e=\Omega(n^2)$ 时，Kruskal算法却比Prim算法好得多。

4.7 多机调度问题

多机调度问题要求给出一种作业调度方案，使所给的n个作业在尽可能短的时间内由m台相同的机器加工处理完成。

> **约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。**

这个问题是**NP完全问题**，到目前为止还没有有效的解法。对于这一类问题,用**贪心选择策略**有时可以设计出较好的近似算法。

4.7 多机调度问题

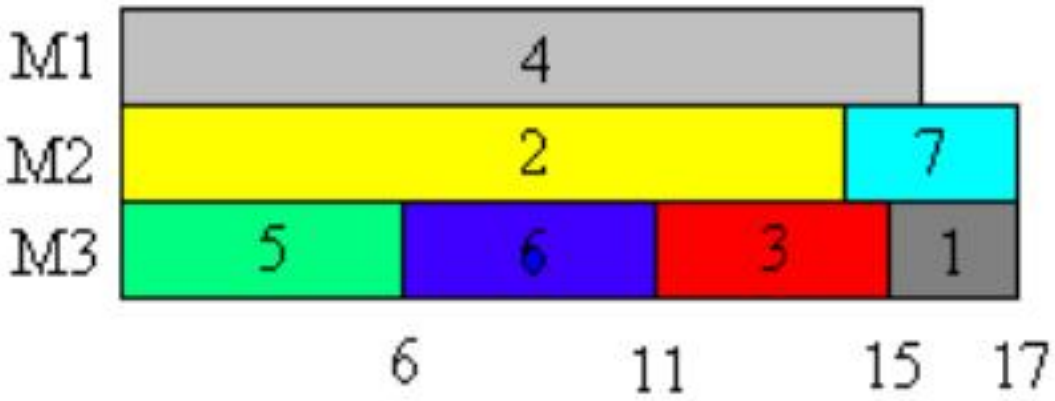
采用**最长处理时间作业优先**的贪心选择策略可以设计出解多机调度问题的**较好的近似算法**。

按此策略，当$n \le m$ 时，只要将机器i的[0, ti]时间区间分配给作业i即可，算法只需要**O(1)**时间。

当 $n > m$ 时，首先将n个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为**O(nlogn)**。

4.7 多机调度问题

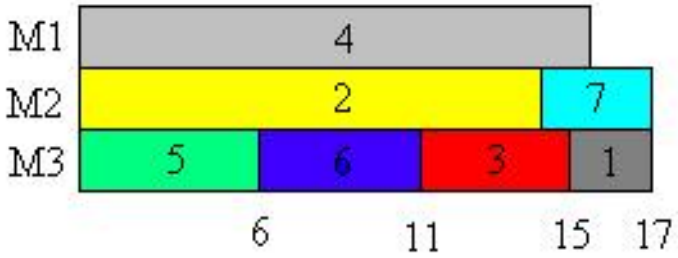
例如，设7个独立作业{1,2,3,4,5,6,7}由3台机器M1，M2和M3加工处理。各作业所需的处理时间分别为{2,14,4,16,6,5,3}。按算法**greedy**产生的作业调度如下图所示，所需的加工时间为17。



上图作为个例实际是最优解了

小任务被排挤了，如何在贪心的基础上考虑小任务？
1、超时加大优先权
2、专门设一个设备

4.7 多机调度问题



作业:{1,2,3,4,5,6,7}　时间:{2,14,4,16,6,5,3}

排序后：{16,14,6,5,4,3,2}

{ 4, 2,5,6,3,7,1}

M1:16　M2:14+3=17　M3:6+5+4+2=17

对所有实例，贪心是不是最优?

4.8 拔河的讨论：贪心与动态规划

小明班里要举行一次拔河比赛，班主任决定将所有人分为两队，每个人都必须参加。两个队伍的体重之和要尽可能相近，当然相同是最好的了。

sample data： 41 23 12 10 9 8 1

模仿上例贪心：

team1： 41 9 1 =51

team2： 23 12 10 8 =53

41+23+12+10+9+8+1=104

team1：41 10 1

team2：23 12 9 8

结论：DP趋近sum/2

4.8 拔河的讨论2

小明班里要举行一次拔河比赛，班主任决定将所有人分为两队，每个人都必须参加。两个队伍的人数之差不能超过1，并且两个队伍的体重之和要尽可能相近，当然相同是最好的了。

```
f[0][0]=1;
//i：对于每个人    //j：体重和【第1约束】  //k：第k个人【第2约束】
for(int i=1;i<=n;i++){
    for(int j=sum/2;j>=a[i];j--){
        for(int k=(n+1)/2;k>=1;k--) {
            f[j][k] = f[j][k] || f[j-a[i]][k-1];
            //只记录能否达到k
        }
    }
}
```

DP最优体现在下面

```
for(int w=sum/2;w>=0;w--)
    if(f[w][(n+1)/2]) {
        cout << w << " " << sum-w << endl;
        break;
    }
```

4.9 贪心算法的习题

一条街的一边有几座房子。因为环保原因居民想要在路边种些树，路边的地区被分割成块，并被编号成1..N，每个部分为一个单位尺寸大小并最多可种一棵树，每个居民想在门前种些树并指定了三个号码B,E,T，这三个数表示该居民想在B和E之间最少种T棵树。当然，B≤E,居民必须记住在指定区不能种多于区域地块数的树，所以T≤E-B+l。居民们想种树的各自区域可以交叉。你的任务是求出能满足所有要求的最少的树的数量。

【输入】

第一行包含数据N，区域的个数

第二行包含H，房子的数目

下面的H行描述居民们的需要:B E T,0

【输出】

输出文件第一行写有树的数目，下面的行包括所有树的位置,相邻两数之间用一个空格隔开。

4.9 贪心算法的习题

【sample input】

9	街道长度
4	4位居民
1 4 2	每位居民的要求
4 6 2	
8 9 2	
3 5 2	

【sample output】

5	几棵树
1 4 5 8 9	位置 （不同答案）

为两边邻居考虑？制造问题
我为人人？相互的
如何开始考虑问题：
从1开始考虑　bottom up

其他例子讨论：
1 5 2
2 3 2

按左边界从小到大排序
（1）如果考虑右边邻居
则先排在4 5，导致2 3 2不满足
（2）如果从最后开始，考虑左边
则排在2 3，1 5 2也满足
3 4 1
1 5 2
按（2）排？

4.9 贪心算法的习题

小伟报名参加中央电视台的智力大冲浪节目，本次挑战赛吸引了众多参赛者，主持人为了表彰大家的勇气，先奖励每个参赛者m元。先不要太高兴！因为这些钱还不一定都是你的！接下来主持人宣布了比赛规则：

首先，比赛时间分为n个时段(n≤500),它又给出了很多小游戏，每个小游戏都必须在规定期限ti前完成(1≤ti≤n)。如果一个游戏没能在规定期限前完成，则要从奖励费m元中扣去一部分钱wi，wi为自然数，不同的游戏扣去的钱是不一样的。当然，每个游戏本身都很简单，保证每个参赛者都能在一个时段内完成，而且都必须从整时段开始。主持人只是想考考每个参赛者如何安排组织自己做游戏的顺序。作为参赛者，小伟很想赢得冠军，当然更想赢取最多的钱！注意:比赛绝对不会让参赛者赔钱！

【输入】

输入文共4行。

第1行为m，表示一开始奖励给每位参赛者的钱；

第2行为n，表示有n个小游戏；

第3行有n个数，分别表示游戏1到n的规定完成期限；

第4行有n个数，分别表示游戏1到n不能在规定期限前完成的扣款数。

【sample input】

```
10000
7
4 2 4 3 1 4 6
70 60 50 40 30 20 10
```

【sample output】

【输出】输出文件仅1行，表示小伟能赢取最多的钱

4.9 贪心算法的习题

【sample input】
10000
7
4 2 4 3 1 4 6
70 60 50 40 30 20 10
【sample output】
9950

贪心原则：最小损失
大问题和小问题的原则是一致的

如何考虑最小损失：大的先排

第2原则/辅助原则？

4.9 贪心算法的习题

贪心使用的总结：

1、排除干扰信息，其中排序具有意义的更少

2、观察可排序的因素，看看那个是符合题意的